machine_learning\m.py

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
import matplotlib.pyplot as plt

df = pd.read_csv("machine_learning/loan_approval_dataset.csv")

df.columns = df.columns.str.strip()

df = df.apply(lambda x: x.str.strip() if x.dtype == "object" else x)

print(df.head())
print(df.shape)
print(df.isnull().sum())
print(df.describe)
print(df.info())

df["education"] = df["education"].map({
    "Graduate": 1,
    "Not Graduate": 0
})

df["self_employed"] = df["self_employed"].map({
    "Yes": 1,
    "No": 0
})

df["loan_status"] = df["loan_status"].map({
    "Approved": 1,
    "Rejected": 0
})

# agar koi NaN bacha ho
df = df.dropna()
x = df.drop("loan_status", axis=1)
y = df["loan_status"]


x_train, x_test, y_train, y_test = train_test_split(
    x, y,
    test_size=0.2,
    random_state=42
```

```
)

#logistic_regession
model = LogisticRegression(max_iter=1000)

model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("confusion_matrix : ", confusion_matrix(y_test, y_pred))

#random_forest_classifier
rf_model = RandomForestClassifier()

rf_model.fit(x_train, y_train)
y_rf_pred = rf_model.predict(x_test)
print("random forest accouracy : ", accuracy_score(y_test, y_rf_pred))
print("confusion matrix : ", confusion_matrix(y_test,y_rf_pred))
print("classfication_metrics : ", classification_report(y_test, y_rf_pred))

#knn_model
knn_model = KNeighborsClassifier()

knn_model.fit(x_train, y_train)
knn_pred = knn_model.predict(x_test)
print("knn : ", accuracy_score(y_test, knn_pred))
print("knn_confusion_matrix : ", confusion_matrix(y_test, knn_pred))
print("classficaton ", classification_report(y_test, knn_pred))

#decision tree

dt_model = DecisionTreeClassifier()

dt_model.fit(x_train, y_train)
dt_pred = dt_model.predict(x_test)
print("dt_accuarcy ", accuracy_score(y_test, dt_pred))
print("dt_confusion matrix ", confusion_matrix(y_test,dt_pred))
print("classfication_matrix ", classification_report(y_test, dt_pred))

#svm
svm = SVC()

svm.fit(x_train, y_train)
svm_pred = svm.predict(x_test)
print("SVM Accuracy:",accuracy_score(y_test, svm_pred))
print("SVM confusion:",confusion_matrix(y_test, svm_pred))
print("SVM classfication :",classification_report(y_test, svm_pred,zero_division=0))

loan_id = int(input("enter the loan id : "))
```

```
no_of_dependemts = int(input("enter the no of dependents : "))

education = input("enter the education (graduate or not graducate : ")

if education == "Graduate":
    education = 1
else:
    education = 0

self_employed = input("enter the self_employed (yes/ no) : ")

if self_employed == "Yes":
    self_employed = 1
else:
    self_employed = 0

income_annum = int(input("enter the income annum : "))
loan_amount = int(input("enter th loan_amount : "))
loan_term = int(input("enter the loan_term : "))
cibil_score = int(input("enter the cibil_score : "))
residential_assets_value = int(input("enter the residentual assets value : "))
commerical_assest_value = int(input("enter the commerical assets value : "))
luxury_assets_value = int(input("enter the luxury assets : "))
bank_assets_value = int(input("enter the bank_assets : "))

new_data = [[
    loan_id,                                #Loan application ka unique number/ID
    no_of_dependemts,                       #Applicant par kitne log depend hain (family
members etc.)
    education,                               #Applicant Graduate hai ya nahi
    self_employed,                          #Khud ka business karta hai ya job karta hai
    income_annum,                            #Saal ki income (annual income)
    loan_amount,                              #Kitna loan maang raha hai
    loan_term,                                 #Loan kitne time ke liye hai (months me)
    cibil_score,                               #credit score, payment history ka score
    residential_assets_value,                  #Ghar/zameen ki value
    commerical_assest_value,                    #Business property/shop etc. ki value
    luxury_assets_value,                         #Car, expensive items etc. ki estimated
value
    bank_assets_value                            #Bank me rakhe assets/savings/investments ki
value
]]

new_data = pd.DataFrame(new_data, columns=x.columns)
predication = rf_model.predict(new_data)

if predication[0] == 1:
    print("your loan is approved by bank ")
else:
    print(" sorry your loan is rejected by bank : ")

```

```
plt.figure(figsize=(6,4))

df["loan_status"].value_counts().plot(kind="bar")

plt.xlabel("loan_status")
plt.ylabel("count")
plt.title("loan approved count")
plt.show()

plt.figure(figsize=(7,5))

plt.scatter(df["income_annum"], df["loan_amount"])
plt.xlabel("Income")
plt.ylabel("loan amount")
plt.title("Income vs loan amount")
plt.show()

plt.figure(figsize=(7,5))

plt.scatter(df["cibil_score"], df["loan_status"])
plt.xlabel("CIBIL Score")
plt.ylabel("Loan Status")
plt.title("CIBIL Score vs Loan Approval")
plt.show()
```